



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
DIVISION OF COMPUTER SCIENCE

Continuous Machine Learning for Cooperative, Connected and Automated Mobility applications

DIPLOMA THESIS

of

GEORGIOS A. KYRIAKOPOULOS

Supervisor: Panayiotis Tsanakas
Professor

Athens, October 2025



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
DIVISION OF COMPUTER SCIENCE

Continuous Machine Learning for Cooperative, Connected and Automated Mobility applications

DIPLOMA THESIS

of

GEORGIOS A. KYRIAKOPOULOS

Supervisor: Panayiotis Tsanakas
Professor

Approved by the three-member scientific committee on the 30th of October 2025

.....
Panayiotis Tsanakas
Professor NTUA

.....
Andreas-Georgios Stafylopatis
Professor Emeritus NTUA

.....
Georgios Alexandridis
Assistant Professor NKUA

Athens, October 2025

.....
Georgios A. Kyriakopoulos
Graduate of School of Electrical and Computer Engineering,
National Technical University of Athens

Copyright © **Georgios A. Kyriakopoulos, 2025.**
All rights reserved.

Copying, storage, and distribution of this work, in whole or in part, for commercial purposes is prohibited. Reproduction, storage, and distribution for non-profit, educational, or research purposes is permitted, provided that the source is cited and this notice is retained. Inquiries regarding the use of this work for commercial purposes should be directed to the author.

The views and conclusions expressed in this thesis represent those of the author and do not necessarily reflect the official positions of the National Technical University of Athens.

Περίληψη

Λέξεις Κλειδιά:

Abstract

Keywords:

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, **Professor Panayiotis Tsanakas**, for giving me the opportunity to conduct my thesis in a field that has been of great personal interest to me.

I would also like to thank **Dr. Georgios Drainakis** and **Dr. Panagiotis Pantazopoulos** of the **I-SENSE Research Group of ICCS/NTUA** for their valuable feedback and guidance towards the completion of this work.

I am grateful to my family and friends for their support and encouragement throughout this important chapter of my life, especially **Valentina** and **Artemis**, as well as **Nick**, **Apostolis**, **Alex**, **Christini**, and **Elina**.

Finally, and most importantly, I would like to thank **Serafeim** and **George**, with whom I shared the vision, the challenges, and the excitement of this work. Their collaboration and commitment were essential in shaping this thesis, and I truly appreciate sharing this journey with them.

Contents

Contents	11
List of Figures	13
List of Tables	15
Extended Abstract in Greek	17
1 Introduction	19
1.1 Problem Statement	19
1.2 Motivation	19
1.3 Thesis Goal	20
1.4 Contributions	20
1.5 Thesis Structure	21
2 Background	23
2.1 Classical Machine Learning	23
2.1.1 Definition and Learning Paradigms	23
2.1.2 Representative Tabular Models	24
2.1.3 Feature Engineering and Pipelines	24
2.1.4 Bias and Variance, Regularization, and Model Selection	24
2.2 Continuous Learning	25
2.2.1 Motivation and Settings	25
2.3 Evaluation: Metrics and Validation	25
2.3.1 Regression Metrics	25
2.3.2 Cross-Validation and Data Leakage	25
2.4 MLOps: Operating ML Systems	26
2.4.1 Scope and Concerns	26
2.5 Concept Drift	26
2.5.1 Definitions and Taxonomy	26
2.5.2 Drift Detector Families	27
2.5.3 Adaptation Strategies	27
2.6 SUMO Simulator	27
2.6.1 Overview, Purpose, and Capabilities	27
2.6.2 OSM → SUMO Pipeline and Scenario Design	28
2.6.3 Outputs Used in This Work	28
2.7 Software Engineering Technologies	28

2.7.1	Containerization with Docker and Docker Compose	28
2.7.2	Backend	29
2.7.3	Data Validation and Schemas	29
2.7.4	APIs and Service Communication (REST/HTTP)	29
2.7.5	Dashboard	29
2.7.6	Columnar Storage: Apache Parquet	29
2.7.7	Artifacts and Reproducibility	29
3	Relevant Work	31
3.1	Tooling Landscape for Drift Detection and Monitoring	31
3.1.1	River	31
3.1.2	scikit-multiflow	31
3.1.3	Alibi and Alibi Detect	32
3.1.4	Evidently	32
3.1.5	NannyML	33
3.2	ETA Prediction: Literature Overview	33
3.3	Novelty and Contributions of the Proposed Platform	35
	Bibliography	42

List of Figures

List of Tables

Extended Abstract in Greek

Chapter 1

Introduction

1.1 Problem Statement

Urban mobility environments are highly dynamic and *non-stationary*, meaning that the statistical properties of traffic data change over time. Machine learning models trained on static historical data often assume a fixed data distribution, an assumption that rarely holds in real city conditions [1]. When deployed in a dynamic urban setting, a model’s performance can gradually degrade as driving patterns shift due to *concept drift*, the evolution of the underlying data relationships over time [2]. This drift is triggered by many factors: for example, road network changes (e.g., construction) can alter traffic flow patterns significantly [3]. Even routine variations like weather and seasonal demand swings can lead to abrupt or gradual shifts in vehicle speeds and congestion levels. A model that performs well on dry, low-traffic days may become less accurate during heavy rain or rush-hour peaks if those conditions were not part of its training distribution [4]. The challenge is that static ML models struggle to maintain accuracy in the face of non-stationarity, as their predictive power deteriorates over time unless the models continually adapt to the evolving urban mobility data [1]. Recognizing and addressing this concept drift in cooperative and automated mobility scenarios is therefore a critical problem identified by the research community.

1.2 Motivation

Given the inevitability of concept drift in real-world intelligent transportation systems, there is a clear need for a practical, drift-aware platform tailored to the mobility domain. While concept drift detection and adaptation have been studied in research, existing solutions are often fragmented or confined to laboratory settings [2, 5]. Current tools lack integration of critical components, for instance, traffic simulation, continuous monitoring, and adaptive retraining are rarely all incorporated into one seamless loop. This means practitioners may detect drifts, but they often have no automated way to replay scenarios, e.g. via simulation, or update models on the fly in response to those drifts [6, 7].

A domain-specific platform would be able to fill this gap by monitoring model performance through time and not just offline metrics, automatically detecting distribution shifts, and triggering model adaptation with minimal human intervention. Such an integrated approach is especially vital in *Cooperative, Connected and Automated Mobility (CCAM)* applications where

conditions can change rapidly and safety is paramount [8]. One key use case is *Estimated Time of Arrival (ETA)* prediction for vehicles, a core service in smart mobility that influences traffic management, logistics, and traveler information [9]. However, many current mobility systems do not continuously recalibrate ETA models as new data, e.g., live traffic feeds or weather updates, arrive [10]. This motivates the creation of a continuous learning platform for mobility: one that integrates traffic simulation to test “what-if” scenarios and generate synthetic data, live performance dashboards to observe errors as they evolve over time, drift detection algorithms to raise alerts when the model performance starts degrading, and an automated retraining pipeline to deploy updated models when needed. By addressing these needs, we aim to ensure that mobility prediction models remain accurate and robust despite the ever-changing urban environment.

1.3 Thesis Goal

The objective of this thesis is to design, implement, and evaluate a microservice-based platform for *continuous machine learning* in urban mobility applications. In essence, the goal is to create a *drift-aware ML platform* that supports end-to-end monitoring and adaptation of predictive models in a city traffic context.

The platform will be built using a modular microservice architecture to ensure scalability and flexibility, where each component (orchestration, model serving, drift detection, etc.) operates as an independent service. Crucially, the platform is intended to be model-agnostic and domain-specific: it can host any machine learning model for mobility prediction tasks, but this thesis will demonstrate and evaluate it primarily through the lens of an Estimated Time of Arrival (ETA) prediction use case. We focus on ETA because of its importance in CCAM scenarios, helping vehicles and travelers coordinate effectively, and because it provides a tangible benchmark to test how well the platform handles concept drift. Upon detecting drift, it will automatically trigger model adaptation, i.e. retraining and model swapping, to self-correct the predictions.

Although ETA is the main case study, the system is designed to accommodate additional models. For example, external predictive modules for Fuel Consumption [11] and Number of Stops [12] will be integrated to showcase that multiple concurrent mobility models can coexist and benefit from the shared drift-aware infrastructure. By the end of this thesis, we aim to show that such a platform can be realized and that it effectively maintains model performance over time in a realistic urban traffic setting.

1.4 Contributions

This thesis makes several contributions to the state of the art in continuous machine learning for mobility:

- **Synthetic Dataset Pipeline:** We developed a reproducible, modular data pipeline that generates training and evaluation datasets using synthetic traffic data from Simulation of Urban MObility (SUMO) [13]. Focusing on a case study of central Athens, the pipeline takes an open-source city road network and creates realistic traffic scenarios (vehicles, routes, events) to produce rich labeled data. This approach ensures that experiments are repeatable and tunable, meaning that the pipeline can be re-run for any city region or traffic pattern by adjusting parameters, thus addressing the shortage of standard drift benchmarks in urban mobility.

- **ETA Prediction Model under Drift:** We designed a feature-engineered ETA prediction model using gradient-boosted decision trees (LightGBM) [14]. The model is trained on the above dataset to estimate travel times for vehicles, and we introduce domain-specific features to improve accuracy. More importantly, the model’s performance is evaluated under simulated drift scenarios, e.g., a sudden shift from normal traffic to heavy rain conditions that reduce vehicle speeds. This allowed us to study how concept drift affects ETA prediction accuracy, and to validate that our model, and platform, can detect and respond to these changes.
- **Time-Lapse Drift Simulation:** We conducted a time-lapse simulation experiment to rigorously assess drift detection and adaptation in an end-to-end fashion. In this setup, the SUMO simulator replays an extended sequence where an initial period of stable conditions is followed by a clear drift event, in this case a transition from dry weather to a storm causing slower traffic. The platform’s drift detectors monitor the real-time prediction error and successfully detect the concept drift when the environment changes. We then measure how the system’s automatic adaptation, triggering model retraining with new data from the rainy period, helps recover the model’s performance. This experiment provides a realistic validation of the platform’s ability to observe, detect, and mitigate drift in a controlled yet lifelike scenario.
- **Drift-Aware Platform Implementation:** We present the design and implementation of a platform for continuous learning in mobility, which is a key practical contribution of this work. The platform includes components for simulation replay that feed historical or synthetic data through the model as if in real time, live dashboarding of model performance metrics, so that users can visualize concept drift as it happens, pluggable drift detection algorithms with calibration tools to set detection thresholds and minimize false alarms, and an automatic retraining and deployment pipeline, which seamlessly replaces the old model with an updated one when drift is confirmed. The entire system is built with modern software frameworks, such as containerized microservices, REST APIs for model serving, and event-driven triggers for retraining, to ensure it can be deployed in real operational environments. By open-sourcing the platform and detailing its architecture, we enable other researchers and practitioners to reproduce our approach and extend it to their own cooperative and automated mobility applications.

1.5 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2: Background** - Introduces the foundational concepts and tools underlying our work. We review machine learning basics relevant to continuous model updating, the functionalities of the SUMO traffic simulator for creating mobility scenarios, the notion of concept drift, such as types of drift and common detection methods, and the platform technologies, like microservices, Docker, and APIs, used to build our solution.
- **Chapter 3: Related Work** - Surveys existing literature and systems in areas pertinent to our thesis. We cover drift detection tools and frameworks, examples being streaming ML libraries and MLOps solutions for model monitoring, examine available urban mobility simulation environments used to study drift, and review prior research on ETA prediction models in intelligent transportation. Finally, we discuss how our approach compares to and

innovates upon the current state of the art.

- **Chapter 4: Dataset and Modeling** - Details our methodology for creating the dataset and developing the ETA prediction model. We describe the data generation process using SUMO for the Athens case study, including how traffic demand patterns and drift scenarios, in this case weather changes, are configured. We then present the ETA model development, including feature engineering, hyperparameter tuning, model selection, and the overall training procedure and the set up of the experiments.
- **Chapter 5: Platform Architecture** - Provides a comprehensive overview of the drift-aware platform's design. We enumerate the microservices and their roles, explain how the system ingests data, either from simulation or future extensions to real streams, how drift detection algorithms are integrated, and how the platform orchestrates retraining and deployment of models. Implementation details of key components, such as the real-time dashboard, are given to illustrate how the system works in practice.
- **Chapter 6: Results** - Presents the results of our evaluation. We report the baseline performance of the ETA model under static conditions and then analyze its behavior under induced drift. We demonstrate the effectiveness of drift detection and quantify the gains from automatic adaptation. The results validate the platform's ability to maintain model accuracy over time.
- **Chapter 7: Conclusions** - Summarizes the contributions and findings of the thesis. We reflect on the success of the continuous learning approach for CCAM applications and any limitations observed. Finally, we outline future research directions, such as improvements to platform scaling and monitoring, or integrating the system with real-time data streams.

Chapter 2

Background

This chapter develops the theoretical and technological background that underpins the thesis. We begin with classical machine learning (paradigms, representative models, and feature pipelines), then introduce continuous/continual learning and evaluation practice, such as metrics and cross-validation. We next cover MLOps concerns for operating ML systems, followed by concept drift definitions, detector families, and adaptation strategies. We then present the SUMO simulator, its scope, scenario construction, OSM→SUMO pipeline, and outputs used, and conclude with the software engineering stack, including Docker/Compose, FastAPI, Dash, Parquet, and REST/HTTP.

2.1 Classical Machine Learning

2.1.1 Definition and Learning Paradigms

Machine Learning (ML) studies algorithms that improve performance on a task T with experience (data) and a performance measure P [15, 16].

Canonical paradigms include:

- **Supervised learning** (labeled examples $\{(x_i, y_i)\}$): learns $f : x \mapsto y$ for regression (e.g., ETA prediction) or classification (e.g., incident detection). Training typically minimizes an empirical loss (e.g., MAE/MSE or cross-entropy) plus regularization to control model capacity and reduce overfitting. For tabular problems, tree ensembles (e.g., gradient boosting) are strong baselines; for spatiotemporal or image data, representation-learning approaches may be preferable.
- **Unsupervised learning** (unlabeled x): discovers structure for exploration, compression, anomaly detection, or feature construction. Typical families include clustering (e.g., k -means), density modeling (e.g., Gaussian mixtures), and dimensionality reduction (e.g., PCA). These can act as pretext steps that enrich supervised models with informative features.
- **Reinforcement learning** (sequential decision-making): learns policies that maximize cumulative reward in an environment. While not central here, RL is relevant to adaptive traffic signal control and routing policies under uncertainty.

2.1.2 Representative Tabular Models

For tabular, heterogeneous, and mixed-scale features common in mobility datasets:

- **Linear models** (OLS, Lasso/Ridge/ElasticNet): as strong baselines; interpretable coefficients and fast training.
- **k-Nearest Neighbors**: non-parametric, local averaging, simple but memory and latency heavy.
- **Decision trees**: axis-aligned splits, handle nonlinearity and interactions.
- **Random Forests**: bagged trees, robust, low-variance and good baseline for tabular.
- **Gradient-Boosted Trees** (XGBoost [17], LightGBM [14], CatBoost [18]): state-of-the-art on many tabular tasks due to additive boosting of weak learners and strong handling of heterogeneity and monotonic constraints.
- **Neural networks** (MLPs): for high-capacity function approximation with feature learning, often require more careful regularization on small tabular sets.

Model choice depends on data size, feature types, latency budgets, and interpretability requirements.

2.1.3 Feature Engineering and Pipelines

End-to-end *pipelines* ensure the same preprocessing is applied in training and inference, preventing leakage and improving reproducibility [19]. Typical steps include:

- **Cleaning**: schema validation, missing-value imputation, outlier handling.
- **Encoding**: one-hot/ordinal for categoricals, frequency/target encoding with care.
- **Scaling**: standardization/robust scaling when models are scale-sensitive.
- **Feature construction**: e.g., temporal (hour-of-day, day-of-week), spatial (segment length, bearings), network features (degree/centrality proxies), and aggregates over short histories.
- **Dimensionality reduction** (PCA) or feature selection for efficiency and generalization.

Packaging these with a library (e.g., scikit-learn **Pipeline**) produces deterministic training and portable inference graphs [20].

2.1.4 Bias and Variance, Regularization, and Model Selection

Practical supervised learning balances *underfitting* (high bias) and *overfitting* (high variance). Core mechanisms include:

- **Regularization**: penalties (e.g., L1/L2), early stopping, dropout, and tree constraints (depth, leaves, learning rate) to control capacity.
- **Model selection**: hyperparameter tuning via cross-validation on training data only; reserve a final hold-out or temporally later set for unbiased estimates.
- **Baselines and sanity checks**: compare to simple heuristics (e.g., historical medians), verify monotonic effects and feature importances, and perform ablations to confirm signal sources.

These practices, combined with leakage-aware pipelines, produce models that generalize and remain interpretable in operational contexts.

2.2 Continuous Learning

2.2.1 Motivation and Settings

Real-world mobility is *non-stationary*: demand cycles, policy changes, incidents, and weather shift the data distribution over time. **Online/continual learning** updates models incrementally, on rolling windows, or via scheduled retraining to preserve performance under change [21, 22]. Practical axes:

- **Update granularity**: per-instance online updates vs. mini-batch vs. periodic retrains.
- **Forgetting mechanism**: sliding windows, decay weights, or explicit resets.
- **Latency/compute budget**: balancing freshness with cost and downtime.

Tooling such as *River* supports streaming estimators, metrics, and change detectors with low-latency, incremental APIs [23].

Algorithms and tooling. In addition to fully online learners, many scikit-learn estimators expose `partial_fit` or warm-start capabilities suitable for mini-batch updates. River provides native streaming models and metrics. Label delay is common in operations, when labels are unavailable, unsupervised monitors on inputs or prediction distributions can provide early signals until supervised error-based detectors activate.

2.3 Evaluation: Metrics and Validation

2.3.1 Regression Metrics

For ETA and related regression tasks:

- **MAE**: $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ (target units). Interpretable and robust to outliers compared to MSE, reads as the average absolute miss in seconds [24].
- **MAPE**: $\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|}$. Scale-free percentage error; unstable when $y \approx 0$ [25].
- **RMSE**: $\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$ (target units). More sensitive to large errors than MAE due to squaring; penalizes outliers heavily.
- **R^2** (coefficient of determination): measures proportion of variance explained by the model, ranging from $-\infty$ to 1; values near 1 indicate strong fit, while $R^2 \leq 0$ means the model performs worse than a constant baseline.

Metric choice reflects operational meaning (e.g., a small absolute error in seconds may be more meaningful than a small percentage error during light traffic) and comparability across scenarios.

2.3.2 Cross-Validation and Data Leakage

Generalization is estimated with cross-validation and matched test scenarios:

- **KFold CV**: partition into k folds, rotate the validation fold, and average scores across iterations [26].
- **StratifiedKFold**: preserves target distribution across folds [27].
- **Temporal splits**: chronological splits for time-ordered evaluation (reported where applicable) to avoid look-ahead.

Preprocessing (e.g., scalers/encoders) is fitted within each training fold and applied to its validation fold; features that would introduce future information are excluded, following standard leakage-avoidance practice [19].

2.4 MLOps: Operating ML Systems

2.4.1 Scope and Concerns

MLOps extends DevOps with data and model-centric practices to make ML systems reliable, reproducible, and observable in practice. Core concerns are [28]:

- **Data management**: explicit schemas, validation checks, drift monitors, and lineage across simulation outputs and features.
- **Experiment tracking & model registry**: hyperparameters, metrics, artifacts, and model versions promoted through stages (dev/staging/prod).
- **CI/CD for ML**: automated training and packaging, reproducible container builds, and controlled rollouts.
- **Monitoring**: service SLOs (latency, availability), data and feature quality, prediction and error distributions, drift signals, and alerting.
- **Governance**: reproducibility, audits, access control to artifacts, and explicit rollback plans.

These address well-documented ML “technical debt” risks (glue code, pipeline entanglement, hidden feedback loops) by favoring modular components, versioned artifacts, and automated checks [28].

2.5 Concept Drift

2.5.1 Definitions and Taxonomy

Concept drift is a change in the joint distribution $P(X, Y)$ over time that invalidates a model trained on historical data [1, 2]. Two major forms include:

- **Virtual/data drift**: $P(X)$ changes while $P(Y | X)$ remains stable (covariate shift).
- **Real/concept drift**: $P(Y | X)$ itself changes (the mapping from features to target shifts).

Temporal archetypes include:

- **Sudden/abrupt**: near-instant change (e.g., road closure).
- **Gradual/incremental**: progressive replacement over time (e.g., evolving driver behavior).
- **Recurring/seasonal**: concepts reappear (e.g., weekday vs. weekend regimes, seasons).

2.5.2 Drift Detector Families

Error-based detectors (supervised). Monitor a streaming error signal (e.g., $|\hat{y} - y|$) and trigger when the mean shifts:

- **Page–Hinkley (PHT):** cumulative sum test sensitive to mean increases [29].
- **DDM/EDDM:** compares on-line error to reference to detect rising error rates (classification focus).

Adaptive window tests. **ADWIN** maintains a dynamically sized window. It repeatedly splits into left/right subwindows and applies a statistical test. If means differ beyond a bound, it cuts the window and signals change [30].

Distributional tests (unsupervised or semi-supervised). Compare recent vs. reference input distributions; e.g., **KSWIN** applies Kolmogorov–Smirnov tests to sliding windows to detect significant changes [31]. These are useful when labels are delayed or unavailable.

Some key trade-offs when choosing a drift detector include sensitivity vs. false alarms, label availability (supervised vs. unsupervised), latency, and computational costs.

2.5.3 Adaptation Strategies

Once drift is suspected/confirmed:

- **Retrain (reset):** fit a fresh model on a recent window of data, good for abrupt regime shifts.
- **Incremental updates:** continue training or add estimators (e.g., more boosting rounds) with recency weighting or forgetting.
- **Windowing:** keep a moving window or buffer to emphasize recent data distribution.
- **Ensembles/repositories:** maintain experts specialized to regimes (e.g., baseline model vs. model for rainy conditions); weight or switch based on current evidence; reuse prior specialists when regimes recur.
- **Safeguards:** trigger throttling, confidence thresholds, or fallbacks (e.g., historical medians) during adaptation.

The choice of adaptation strategy depends on factors like drift speed, severity, data availability, compute budgets, and risk tolerance [1, 2].

2.6 SUMO Simulator

2.6.1 Overview, Purpose, and Capabilities

Eclipse SUMO (Simulation of Urban MObility) is an open-source, microscopic, multi-modal traffic simulator developed by the German Aerospace Center (DLR) and now under the Eclipse ecosystem. It is widely used in research and practice for modeling vehicle-level dynamics, routing, signals, and emissions across large networks [32, 33]. “Microscopic” means each vehicle is an agent governed by car-following (for example, the Krauss model [34]) and lane-changing models, enabling fine-grained phenomena, such as queues, spillback and shockwaves.

Key capabilities include:

- **Network handling:** import, edit, and simulate large road graphs with multiple lane attributes, priorities, and traffic signal programs.
- **Demand modeling:** trips, routes, O/D flows, public transport; dynamic routing and rerouting.
- **Control & APIs:** TraCI [35] (TCP control interface) for online interaction; Python clients to step the simulator, inject vehicles, or read states.
- **Scenarios:** controlled experiments (e.g., incidents, weather surrogates, signal changes) with reproducibility via configuration files.

2.6.2 OSM → SUMO Pipeline and Scenario Design

A typical path to realistic scenarios is converting **OpenStreetMap** (OSM) into a SUMO network:

1. Obtain OSM data for the area of interest.
2. Convert with **netconvert** (infers edges, lanes, priorities, optionally traffic lights) [36, 37].
3. Refine with **netedit** (GUI) for manual adjustments and optionally use **polyconvert** for shapes or polygons.
4. Generate demand via **randomTrips.py**, route assignment, or O/D matrices.

This pipeline yields geographically faithful networks (e.g., central Athens) while preserving control over demand, signal plans, and experiment design [33].

2.6.3 Outputs Used in This Work

SUMO produces multiple outputs consumable by analytics/ML:

- **Floating Car Data (FCD):** per-step vehicle states (id, position, speed, angle, lane, etc.) via `--fcd-output`. FCD resembles GPS traces and is well-suited for trip assembly and ETA features [38].
- **Emissions/Fuel** via `--emission-output`: per-vehicle per-step estimates for fuel consumption and pollutants, with documented units and models [39].
- Additional: **tripinfo** (per-trip travel times), edge/lane aggregations, detector outputs, and more [33].

Outputs are XML by default, with support for additional formats like CSV and Parquet available. The XML format can be converted to CSV for efficient downstream processing, with provided tools like `xml2csv`.

2.7 Software Engineering Technologies

2.7.1 Containerization with Docker and Docker Compose

Docker packages code, runtime, and dependencies into portable container images that run consistently across environments [40, 41]. The benefits of using it include:

- Reproducible builds and isolated dependencies.

- Fast startup and easy horizontal scaling via replicas.
- Immutable artifacts that enable safe rollbacks.

Docker Compose declaratively defines multi-container applications (services, networks, volumes) and brings them up with a single command, ideal for local development or single-host deployments of microservices [40].

2.7.2 Backend

FastAPI is a high-performance Python framework for REST APIs with type-hinted models, validation (Pydantic [42]), and autogenerated OpenAPI/Swagger docs [43]. It suits prediction endpoints, orchestration commands (e.g., retraining triggers), and health checks.

2.7.3 Data Validation and Schemas

Schemas enforced at API boundaries and in dataframes reduce runtime errors and prevent silent corruption. Typed request/response models (e.g., Pydantic) and dataframe-level checks (e.g., declared dtypes, value ranges, categorical domains) provide early failure and clearer contracts between services.

2.7.4 APIs and Service Communication (REST/HTTP)

Services interact over **HTTP** following **REST** principles (uniform interface, statelessness, resource orientation) [44]. Conventions include:

- Clear resource paths (e.g., `/predict`, `/models/current`, `/drift/status`).
- JSON payloads with explicit schemas; versioned endpoints.
- Proper status codes (2xx success, 4xx client errors, 5xx server errors) and idempotency where applicable.

Within Docker networks, services resolve each other by name; an API gateway or reverse proxy can front external access. OpenAPI contracts facilitate testing and integration.

2.7.5 Dashboard

Dash builds interactive dashboards in Python using Plotly components, enabling real-time monitoring (metrics, error traces, drift alarms) and control widgets (e.g., start/stop simulation) without JavaScript [45].

2.7.6 Columnar Storage: Apache Parquet

Parquet is a columnar format optimized for analytical IO: selective column reads, strong compression, and explicit schemas. It integrates with PyArrow and Pandas and supports large, append-friendly datasets typical in streams and simulations [46, 47].

2.7.7 Artifacts and Reproducibility

Models and datasets should be versioned and stored as immutable artifacts with metadata (schema, training data window, metrics). Deterministic serialization (e.g., joblib) and pinned

environments support reproducible inference; storing artifacts in object storage enables auditing and rollback.

Chapter 3

Relevant Work

3.1 Tooling Landscape for Drift Detection and Monitoring

3.1.1 River

River is an open-source Python library for online machine learning on streaming data, born from the merger of the Creme and scikit-multiflow projects. It provides a single unified framework to train models incrementally, one sample at a time, which makes it well-suited to scenarios with continuous data and potential concept drift. River includes a variety of learning algorithms (e.g. linear models, Hoeffding trees, ensemble methods) that can update their parameters on the fly, as well as evaluators and metrics that update incrementally along with the model. Notably, River treats concept drift as a first-class concern: it implements plug-and-play drift detection methods like ADWIN, DDM, and EDDM to signal when the statistical properties of the data or the model’s error rate have changed significantly. A major strength of River is its efficiency and low overhead for streaming applications, it avoids expensive retraining by doing continuous learning, enabling rapid adaptation to changing data. However, River’s focus on single-instance updates means it may not directly leverage batch acceleration or GPU computing as efficiently as some batch frameworks. Moreover, building deep neural models in River is less common, so for very complex patterns (e.g. image or graph data) one might need external solutions. Overall, River is focused on streaming adaptation and concept drift handling, making it a powerful tool for real-time ML systems[23].

3.1.2 scikit-multiflow

scikit-multiflow is an earlier Python framework for stream learning and concept drift, which was one of River’s predecessors (Montiel et al., 2018)[48]. Inspired by the MOA stream mining software in Java, scikit-multiflow brought popular stream learning algorithms and evaluation tools into the Python ecosystem. It supports a range of tasks including classification, regression, and even multi-output prediction, and it provides several drift detection algorithms (e.g. ADWIN, DDM, EDDM, Page-Hinkley) as part of its toolkit. A key design goal was compatibility with scikit-learn, allowing users to integrate stream learning with familiar APIs. The strengths of scikit-multiflow include its breadth of implemented methods (many inherited from the MOA framework) and the ability to simulate data streams with built-in generators for controlled concept drift (e.g. abrupt or gradual drift in synthetic data). One limitation, however, is that scikit-multiflow is no longer as actively maintained; its functionality has effectively been subsumed into River, which

offers a more consolidated and improved interface. In practice, new projects prefer River for streaming tasks, but scikit-multiflow remains a reference point in the literature and is still useful for benchmarking new drift detection techniques due to its comprehensive implementation of classical methods.

3.1.3 Alibi and Alibi Detect

Alibi is an open-source library by Seldon Dev focused on machine learning model inspection and explainability, offering techniques like counterfactual explanations and influence scores for black-box models (e.g. explaining why a prediction was made)[[AlibiExplainability](#)]. Building on this, the Seldon team also released Alibi Detect, which is a library specifically for outlier detection and concept drift detection in ML applications [github.com](#). Alibi Detect provides a collection of both offline and online detectors for various data types: tabular data, text, images, and time series [github.com](#). It includes statistical methods (like Kolmogorov-Smirnov or Chi-square tests for distribution drift in features) as well as learned detectors (like drift detection using neural network embeddings for complex data such as images) [github.com](#). A notable focus of Alibi Detect is support for different modalities and integration with modern ML stacks: it can use TensorFlow or PyTorch under the hood, which allows it to implement advanced detectors (e.g. a PCA-based outlier detector or a classifier-based drift detector) that leverage GPUs. The strength of Alibi/Alibi Detect lies in its broad coverage: it tackles not just drift but also outliers and even adversarial example detection in one package, making it a flexible choice for production monitoring where explainability and data quality are needed alongside drift detection [github.com](#). One limitation is that Alibi Detect is a lower-level library – it provides algorithms but not a full monitoring solution; users must integrate it into their workflow (for example, scheduling periodic drift checks or responding to detector alerts in an MLOps pipeline). Additionally, configuring the more complex detectors may require expert knowledge (for instance, setting thresholds or choosing embedding models). In summary, Alibi (for explainability) and Alibi Detect (for drift/outliers) offer a robust toolkit focused on model insight and data drift, well-suited for use cases where understanding model behavior and detecting distribution shifts are both important (Looveren et al., 2020)[49].

3.1.4 Evidently

Evidently is an open-source Python library and toolset designed to evaluate, test, and monitor machine learning models and data in production [medium.com](#). It provides pre-built reports and dashboards to analyze things like data drift, target drift, data quality, and model performance over time [medium.com](#) [medium.com](#). One of Evidently’s core features is the ability to generate an interactive HTML dashboard comparing a reference dataset (for example, the training data or last week’s data) to a current dataset; the dashboard includes visualizations and statistical tests that highlight any significant changes in feature distributions or model outputs. Under the hood, Evidently employs statistical drift detection metrics suitable for each feature type (e.g., Kolmogorov–Smirnov test for continuous features, Jensen-Shannon divergence or chi-square for categorical features) [evidentlyai.com](#) [medium.com](#). It also tracks model performance metrics (if ground truth is available) or proxy metrics (like prediction score distributions) as part of its monitoring reports. The strengths of Evidently include its ease of use and rich visualization – with minimal code, a user can produce a comprehensive report that would otherwise require manual analysis. This makes it a popular choice for integrating into Jupyter notebooks or automated pipelines to regularly check for data drift and model decay. Evidently also supports customizable

test suites: users can define assertions (e.g., “no feature drift beyond a certain threshold”) and get alerts when these are violated [medium.com](#) . A current limitation is that Evidently’s real-time capabilities are evolving; it was initially built for batch analysis and reporting. Using it in a truly live monitoring scenario (continuous streaming data) may require additional effort (such as writing a job to compute metrics on rolling windows). Additionally, while it identifies drift and performance issues, it doesn’t automatically retrain models or suggest fixes – it’s primarily a monitoring dashboard. In short, Evidently fills the role of ML monitoring and data validation by making drift detection and model quality checks more accessible and transparent (Evidently AI, 2023)[50].

3.1.5 NannyML

NannyML is a newer open-source library specifically focused on post-deployment model monitoring, with an emphasis on detecting concept drift and estimating model performance without immediate ground truth (Grootendorst et al., 2023)[**NannyML**]. It aims to answer the question: “How is my model performing now, given that I might not have labels for the latest predictions?” To this end, NannyML provides tools for data drift detection (monitoring changes in input feature distributions) as well as algorithms to infer performance drop. A centerpiece is its Confidence-Based Performance Estimation (CBPE) method, which uses the model’s prediction probabilities and detected data drifts to estimate metrics like accuracy or ROC AUC in production before real labels arrive [dev.to](#) . In practical terms, NannyML can alert you that “your model’s inputs have shifted significantly, and it estimates your model’s accuracy has dropped from (say) 0.85 to 0.70” – allowing proactive retraining or investigation [dev.to](#) . Another innovative feature is the “reverse drift” (RCD) concept: assessing how changes in input data would be expected to affect the target distribution and thus the model’s error, essentially linking drift to business impact [nannyml.com docs.nannyml.com](#) . The strength of NannyML lies in this performance-centric approach to drift: it goes beyond flagging that data has changed, by quantifying the likely effect on model predictions and outcomes. This is particularly useful in industries where obtaining true labels is slow or expensive (e.g., credit risk, where you only know defaults after many months). NannyML’s limitations include that it currently supports primarily classification tasks (binary and multiclass) for its performance estimation module [nannyml.readthedocs.io](#) . The techniques can be complex to calibrate – for instance, CBPE assumes the model’s probability estimates are well-calibrated and that past relationships hold, which might not always perfectly predict actual performance. Additionally, as a specialized library, it may need to be integrated into an existing MLOps setup (it provides Python APIs and some UI components, but not a full monitoring server on its own). In summary, NannyML represents an advanced approach to concept drift monitoring: instead of just detecting drift, it focuses on what that drift means for model accuracy, helping close the gap between drift signals and model maintenance actions [dev.to](#) .

3.2 ETA Prediction: Literature Overview

Predicting travel time or estimated time of arrival (ETA) is a well-studied problem in the transportation domain, and a variety of machine learning approaches have been explored. Early works approached ETA prediction with classical time-series models like ARIMA, treating travel time as a time-dependent process; while such statistical models worked in limited settings, they struggled with the inherent non-linearities and context dependencies of traffic data [frontiersin.org](#) . Over the past decade, the field has seen a surge of deep learning methods that capture complex spatial and temporal patterns in traffic networks. For example, recurrent neural

networks (RNNs) and long short-term memory (LSTM) architectures have been used to model vehicle trajectories as sequences of GPS points, successfully learning temporal dependencies such as road segment travel times and rush-hour effects (Duan et al., 2016; Shen et al., 2019)[51, 52]. Convolutional neural networks have been applied to encode spatial information — one notable approach is to treat a path’s GPS sequence like an “image” or matrix (e.g., via a grid or along-route segmentation) and use CNNs to extract features (Wang et al., 2018)[53]. More recently, attention mechanisms and transformers have been introduced: AttentionTTE by Li et al. (2024) employs self-attention to capture global interactions among road segments, combined with a recurrent module for local temporal trends [frontiersin.org](#) [frontiersin.org](#) . This achieved state-of-the-art results on a large trajectory dataset, indicating the benefit of modeling both local and long-range dependencies in routes [frontiersin.org](#) [frontiersin.org](#) . Similarly, researchers have explored multi-task learning in deep models – for instance, the WDR (wide-deep-recurrent) model and CoDriver system incorporated an auxiliary task (learning driver behavior style) to improve ETA predictions, demonstrating that additional contextual signals can improve accuracy [frontiersin.org](#) .

In parallel to sequence-based methods, graph-based approaches have gained prominence for ETA and traffic prediction. In a road network, intersections and road segments can be naturally represented as nodes and edges in a graph, so graph neural networks (GNNs) can explicitly model the connectivity and traffic flow between road segments. Derrow-Pinion, Xu, Layne, and al. [54] describe a GNN-based ETA model deployed in production for Google Maps, which learns from massive amounts of traffic data while incorporating road network topology and dynamic conditions (like accidents or rush hour patterns). By using GNN layers to propagate information along the road graph, their model can account for upstream slowdowns or congestion that classical models might miss [arxiv.org](#) . This graph-based ETA estimator yielded significant improvements in real-world accuracy – for example, reducing large ETA errors by over 40% [arxiv.org](#) . Other studies have followed similar ideas, building spatio-temporal graph models that combine GNNs with temporal sequence modeling (e.g. graph convolution plus LSTM) to predict travel times under varying conditions (Jia et al., 2021; Guo et al., 2022)[55, 56]. These advanced deep learning models represent the current state-of-the-art, especially when rich data is available (historical trajectories, real-time traffic sensors, etc.). They tend to excel at capturing complex patterns, but at the cost of higher complexity and data requirements.

Despite the success of deep neural approaches, tabular data methods remain highly relevant, especially in industry settings where interpretability, simplicity, or limited data are considerations. Many practical ETA prediction systems reduce the problem to a regression on engineered features: for example, features might include the route distance, expected traffic speed, number of traffic lights, time of day, day of week, weather conditions, and so on. Gradient boosting decision trees (GBDT) have been a popular choice for such tabular formulations (Guin, 2006; Zhang and Haghani, 2015)[57, 58]. GBDT models (implemented in tools like XGBoost or LightGBM) offer strong performance on structured data and have the advantage of producing feature importance insights, which is valuable for understanding ETA drivers. Zhang and Haghani (2015) demonstrated that with carefully constructed input features (including real-time traffic indices), a GBDT model can achieve accuracy comparable to more complex models, though it may need frequent updating to handle drift [frontiersin.org](#) . In recent work, Antypas et al. (2022)[59] applied XGBoost, CatBoost, and LightGBM to predict bus arrival times in a smart city context, showing that these boosted tree models are competitive for ETA and can be tuned to high accuracy given domain-specific feature engineering. In their case, gradient boosting achieved around 30% [researchgate.net](#) . The continued popularity of gradient boosting in this domain is due

to several factors: (i) Tabular features can encode a lot of prior knowledge (for instance, the road speed limit or historical average travel time on a segment is an informative predictor that a tree can easily use); (ii) training and inference are fast and resource-efficient compared to deep sequence models, which is useful for deployment on edge devices or with limited infrastructure; and (iii) the models are easier to maintain (one can retrain a new boosted tree model on recent data periodically, and examine feature importances if the model starts to degrade). Of course, a limitation is that these models do not automatically capture sequential or spatial relationships unless those are manually turned into features. Therefore, the current trend in ETA research and applications is often a hybrid one: use domain knowledge to engineer strong features and/or combine outputs of simpler models, and, where possible, incorporate the structure of the problem (sequential or graph) via specialized models or ensemble approaches. Well-known benchmarks, such as the NYC Taxi trip duration dataset (Kaggle 2015) or the Google Maps ETA data (used in CIKM 2021 Challenge), often show that gradient boosting with rich features can be very effective, coming close to deep learning methods when the latter are not heavily optimized. In summary, ETA prediction methods span from deep learning models leveraging spatio-temporal structure to gradient-boosted tree models on tabular features, and the choice often depends on available data and the need for interpretability versus maximum accuracy.

3.3 Novelty and Contributions of the Proposed Platform

The platform developed in this thesis brings together ideas from concept drift research, MLOps, and domain-specific traffic simulation to create a closed-loop adaptive learning system for mobility. Here we summarize the key novel contributions of this platform, and how it differentiates itself from existing tools and literature:

1. **Controlled Drift Simulation with SUMO:** In order to study concept drift in a realistic yet controlled manner, we integrate a traffic simulation (SUMO – Simulation of Urban MObility) into the platform. SUMO allows us to generate synthetic traffic data under varying conditions with fine control. A novel aspect is the creation of a time-lapse scenario in which we deliberately introduce a sudden change in environment: for example, the platform simulates 10 hours of normal driving conditions followed by 10 hours of heavy rain, where rain is modeled by globally reducing the road friction coefficient in SUMO. This causes vehicles to slow down and drive differently, thereby inducing concept drift in the input data and the relationship between features (e.g. speeds, distances) and targets (travel time, etc.). By having this capability to manufacture drift on demand, we can evaluate how well different detectors and adaptation strategies perform on a known “ground truth” drift event. Traditional drift detection research often relies on either real-world events (which are unpredictable) or synthetic data generators for drift (e.g., rotating hyperplane datasets) jmlr.csail.mit.edu. Our use of SUMO bridges that gap by producing realistic, domain-specific drift: it’s not just abstract changes in data distribution, but changes grounded in a physical scenario (weather change in traffic) that a transportation agency would care about. This approach is relatively unique – to our knowledge, few if any published works have demonstrated a full ML workflow where a traffic microsimulator is used to inject controlled drifts and then automatically detect and counteract them in real time. Moreover, the platform continuously tracks the model’s performance during the simulation; as the rain scenario unfolds, we log the model’s prediction error metrics (e.g. mean absolute error for ETA) to observe how performance degrades due to drift. This provides an authentic demonstration of concept drift’s impact on an ML model deployed in a dynamic environment.

2. Multi-Task Prediction and Monitoring: The platform is designed to handle multiple related prediction tasks simultaneously, which is a departure from most drift detection case studies that focus on a single prediction output. In our implementation, a single model (or a set of coordinated models) produces predictions for ETA, fuel consumption, and number of stops for a given vehicle trip. This multi-task setup reflects a more complex operational scenario (for instance, a smart mobility service might want to predict not just arrival time but also fuel usage and stops for logistics planning). The contribution here is twofold: (a) from a modeling perspective, the system can either use multi-output regression models or separate models that share features – either way, it must manage and monitor several predictions at once; and (b) from a drift detection perspective, the platform monitors drift on multiple data streams and multiple performance metrics concurrently. For example, during the rain-induced drift, ETA predictions might degrade first while fuel predictions remain stable, or vice versa, so our system watches each task. This is novel compared to existing tools like River or NannyML which typically assume a single target. Our platform demonstrates how a drift in the environment can have different manifestations across tasks (e.g., rain might drastically affect ETA and fuel consumption but not the predicted number of stops if stops depend more on route layout than speed). It required developing a custom monitoring strategy that can aggregate signals from multiple drift detectors and multiple outputs. We implemented a simple consensus mechanism: the drift detection micro-service in our system runs detectors on input features (data drift) and on each model’s error stream (performance drift), and only if a consistent drift signal is observed (for instance, error rate for ETA spiking and a known change in input distribution) do we declare a drift event and trigger adaptation. This reduces false alarms in a noisy multi-task setting. Supporting multiple tasks in one platform is a practical contribution because real-world systems often have to monitor dozens of metrics – our work provides a template for how to achieve that in an automated way.

3. Automated Model Retraining and Hot-Swapping: Perhaps the most significant contribution of the platform is the seamless closed-loop adaptation capability. When concept drift is detected (either via a statistical test or a drop in performance beyond a threshold), the platform doesn’t just raise an alert – it initiates a retraining pipeline and deployment of an updated model, without human intervention. Concretely, the system maintains a buffer of recent data (e.g., the last 1 hour of vehicle trips) and when drift is confirmed, it triggers a retraining job that fine-tunes or re-fits the ML model using this new data distribution. We implement this efficiently for the gradient boosting model used for ETA: since training from scratch can be done quickly on the updated dataset (which is not huge in our case), we opted for a simple re-training rather than online learning updates, to ensure we capture the new pattern (e.g., slower speeds in rain) comprehensively. Once the new model is trained and validated on a hold-out set, the platform performs a hot swap: the running system begins using the new model for all subsequent predictions, with minimal downtime. Techniques from continuous deployment are used here – for example, the old model and new model might briefly run in parallel to ensure the new one is warmed up, and then a switch is flipped (this can be done via a REST API call to the predictor service to load the new model). This kind of automated retraining and deployment is often discussed in MLOps but not commonly demonstrated end-to-end in academic literature researchgate.net researchgate.net . Recent work by Abhay et al. (2025)[60] proposed an automated pipeline using Evidently to detect drift and trigger retraining, deploying the new model if it outperforms the old – our platform realizes a similar idea, but in the specific context of mobility and streaming data, and combines it with the aforementioned drift simulation. The novelty here is showing the entire loop: we not only detect drift, we close the loop by adapting to it immediately. This required solving engineering challenges (e.g., ensuring that the system can retrain while continuing to serve

predictions, avoiding a halt) and design challenges (deciding when retraining is worth it to avoid model churn). The outcome is an architecture where model performance degradation is not only observed but automatically countered by an update – moving towards the ideal of a self-healing ML system.

4. Real-Time Dashboard and Visualization: To make the system’s operations interpretable and to aid in human-in-the-loop oversight, the platform includes a real-time dashboard that visualizes the current state of the simulation and the ML models. This dashboard (built with Dash/Plotly) displays key information such as: current traffic conditions in the simulation (e.g., a map of Athens with vehicles, possibly highlighting rain conditions), the live predictions vs. ground truth for ETA and other tasks, and drift detection metrics over time (for instance, a timeline showing the p-value of a drift test or the model’s error rate). When a drift is detected and a model swap occurs, the dashboard visibly flags this event (e.g., “Drift detected at 10:30, model retrained and updated at 10:35”), and subsequent improvements in error can be observed. While a dashboard itself is not a novel research contribution, integrating it end-to-end with the platform is a valuable demonstration for practitioners. It closes the feedback loop by allowing users to watch the ML system adapt in real time, thereby building trust. Many existing drift tools like Evidently produce static reports or require the user to inspect logs; in contrast, our system’s UI shows a continuously updating view, which is especially important in a streaming context. This also helps communicate the results: for example, stakeholders can see that during the rain scenario the original model’s error kept rising, but as soon as the retrained model was deployed, the error drops back to normal levels – effectively illustrating the benefit of the adaptive approach.

In summary, the platform distinguishes itself by combining drift generation, detection, and adaptation in one unified environment. Unlike existing monitoring libraries that stop at detection, or online learning frameworks that update models but often on synthetic data, our system covers the full lifecycle on a realistic traffic problem: it manufactures a drift (rainy weather) in a mobility digital twin, observes the model’s decline, catches it via multiple detectors, and then remedies it through automated retraining – all while delivering insights through a real-time interface. This end-to-end closed-loop approach, especially applied in the context of cooperative, connected, and automated mobility, is a novel contribution demonstrating how advanced MLOps techniques (continuous monitoring, automatic retraining) can be applied to maintain model performance in non-stationary environments. It effectively serves as a prototype for next-generation intelligent traffic management systems where AI models remain reliable over time by continually learning from new conditions. The lessons from this platform could be extended to other domains as well, but are particularly pertinent to mobility where change (in weather, human behavior, infrastructure) is constant and must be managed by adaptive learning systems.

Bibliography

- [1] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. “A survey on concept drift adaptation”. In: *ACM Computing Surveys (CSUR)* 46.4 (2014), pp. 1–37.
- [2] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., and Zhang, G. “Learning under concept drift: A review”. In: *IEEE Transactions on Knowledge and Data Engineering* 31.12 (2018), pp. 2346–2363.
- [3] Maheshwari, H., Yang, L., and Pazzi, R. W. “Machine Learning Advancements in Urban Traffic Simulation: A Comprehensive Survey”. In: *IEEE Open Journal of Intelligent Transportation Systems* 6 (2025), pp. 1027–1052.
- [4] Baier, L., Hofmann, M., Kühl, N., Mohr, M., and Satzger, G. “Handling Concept Drifts in Regression Problems—the Error Intersection Approach”. In: *arXiv preprint arXiv:2004.00438* (2020).
- [5] Greco, S., Vacchetti, B., Apiletti, D., and Cerquitelli, T. “Unsupervised Concept Drift Detection from Deep Learning Representations in Real-time”. In: *arXiv preprint arXiv:2406.17813* (2025).
- [6] Xin, H., Zhang, X., Tang, R., Yan, S., Zhao, Q., Yang, C., Cui, W., and Yang, Z. “LitSim: A Conflict-aware Policy for Long-term Interactive Traffic Simulation”. In: *arXiv preprint arXiv:2403.04299* (2024).
- [7] Modesto, C., Borges, J., Nahum, C., Matni, L., Both, C. B., Cardoso, K., Gonçalves, G., Correa, I., Lins, S., Silva, A., and Klautau, A. “Towards a Robust Transport Network With Self-adaptive Network Digital Twin”. In: *arXiv preprint arXiv:2507.20971* (2025).
- [8] ERTRAC Working Group “Connectivity and Automated Driving”. *Connected, Cooperative and Automated Mobility Roadmap*. Tech. rep. Version 10. European Road Transport Research Advisory Council (ERTRAC), 2022.
- [9] Abdi, A. and Amrit, C. “A Review of Travel and Arrival-Time Prediction Methods on Road Networks: Classification, Challenges and Opportunities”. In: *PeerJ Computer Science* 7 (2021), e689.
- [10] Arifi, A., Bouros, P., and Chondrogiannis, T. “A Study on ETA Prediction using Machine Learning and Recovered Routes”. In: *Proceedings of the Workshops of the EDBT/ICDT 2024 Joint Conference*. CEUR Workshop Proceedings. Paestum, Italy: CEUR-WS.org, 2024.
- [11] Angelis, G. *Fuel Consumption Prediction Model*. Unpublished work, integrated in drift-aware platform. 2025.
- [12] Tzelepis, S. *Number of Stops Prediction Model*. Unpublished work, integrated in drift-aware platform. 2025.
- [13] Krajzewicz, D., Erdmann, J., Behrisch, M., and Bieker, L. “Recent development and applications of SUMO - Simulation of Urban MObility”. In: *International Journal On Advances in Systems and Measurements* 5.3&4 (2012), pp. 128–138.

- [14] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. 2017, pp. 3146–3154.
- [15] Murphy, K. P. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012. URL: <https://www.cs.ubc.ca/~murphyk/MLbook/pml-toc-1may12.pdf>.
- [16] Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006. URL: <https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/>.
- [17] Chen, T. and Guestrin, C. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785. URL: <https://arxiv.org/abs/1603.02754>.
- [18] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. “CatBoost: Unbiased Boosting with Categorical Features”. In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. 2018, pp. 6638–6648. URL: <https://arxiv.org/abs/1706.09516>.
- [19] *Metrics and scoring: quantifying the quality of predictions*. https://scikit-learn.org/stable/modules/model_evaluation.html. Accessed 2025-10-25.
- [20] *Pipeline: chaining estimators — scikit-learn*. <https://scikit-learn.org/stable/modules/compose.html>. Accessed 2025-10-25.
- [21] Shalev-Shwartz, S. “Online Learning and Online Convex Optimization”. In: *Foundations and Trends in Machine Learning* 4.2 (2012), pp. 107–194. DOI: 10.1561/22000000018. URL: <https://www.cs.huji.ac.il/~shais/papers/OLsurvey.pdf>.
- [22] Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., and Wermter, S. “Continual Lifelong Learning with Neural Networks: A Review”. In: *Neural Networks* 113 (2019), pp. 54–71. DOI: 10.1016/j.neunet.2019.01.012. URL: https://www2.informatik.uni-hamburg.de/wtm/publications/2019/PKPKW19/Parisi-2019_Continual-lifelong-learning-with-NN-review.pdf.
- [23] Montiel, J., Halford, M., Mastelini, S. M., Bolmier, G., Sourty, R., Vaysse, R., Zouitine, A., Guillet, F., Read, J., Abdessalem, T., and Bifet, A. “River: Machine Learning for Streaming Data in Python”. In: *Journal of Machine Learning Research* 22.110 (2021), pp. 1–8. URL: <https://www.jmlr.org/papers/v22/20-1380.html>.
- [24] *mean_absolute_error — scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html. Accessed 2025-10-25.
- [25] *mean_absolute_percentage_error — scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_percentage_error.html. Accessed 2025-10-25.
- [26] *Cross-validation: evaluating estimator performance — scikit-learn*. https://scikit-learn.org/stable/modules/cross_validation.html. Accessed 2025-10-25.
- [27] *StratifiedKFold — scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html. Accessed 2025-10-25.
- [28] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. “Hidden Technical Debt in Machine Learning Systems”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015, pp. 2503–2511. URL: <https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>.

- [29] Page, E. S. “Continuous Inspection Schemes”. In: *Biometrika* 41.1-2 (1954), pp. 100–115. DOI: 10.1093/biomet/41.1-2.100. URL: <https://academic.oup.com/biomet/article-abstract/41/1-2/100/456627>.
- [30] Bifet, A. and Gavaldà, R. “Learning from Time-Changing Data with Adaptive Windowing”. In: *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*. 2007, pp. 443–448. DOI: 10.1137/1.9781611972771.42. URL: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611972771.42>.
- [31] Raab, C., Heusinger, M., and Schleif, F.-M. “Reactive Soft Prototype Computing for Concept Drift Streams”. In: *Neurocomputing* 398 (2020). KSWIN, pp. 309–327. DOI: 10.1016/j.neucom.2020.02.119. URL: <https://arxiv.org/pdf/2007.05432>.
- [32] Alvarez Lopez, P., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flöttmann, Y.-P., Hilbrich, A., Lücken, L., Rummel, J., Wagner, P., and Wießner, E. “Microscopic Traffic Simulation using SUMO”. In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. 2018. DOI: 10.1109/ITSC.2018.8569938. URL: <https://elib.dlr.de/127994/1/08569938.pdf>.
- [33] *Eclipse SUMO Documentation (Overview/Output)*. <https://sumo.dlr.de/docs/>. Accessed 2025-10-25.
- [34] Krauß, S. “Microscopic modeling of traffic flow: Investigation of collision free vehicle dynamics”. PhD thesis. DLR, 1998.
- [35] Wegener, A., Piórkowski, M., Raya, M., Hellbrück, H., Fischer, S., and Hubaux, J.-P. “TraCI: an interface for coupling road traffic and network simulators”. In: *Proceedings of the 11th Communications and Networking Simulation Symposium*. 2008, pp. 155–163.
- [36] *netconvert — SUMO*. <https://sumo.dlr.de/docs/netconvert.html>. Accessed 2025-10-25.
- [37] *Networks: Import from OpenStreetMap — SUMO*. <https://sumo.dlr.de/docs/Networks/Import/OpenStreetMap.html>. Accessed 2025-10-25.
- [38] *Simulation Output: FCD Output — SUMO*. <https://sumo.dlr.de/docs/Simulation/Output/FCDOutput.html>. Accessed 2025-10-25.
- [39] *Simulation Output: EmissionOutput — SUMO*. <https://sumo.dlr.de/docs/Simulation/Output/EmissionOutput.html>. Accessed 2025-10-25.
- [40] *What is Docker?* <https://docs.docker.com/get-started/overview/>. Accessed 2025-10-25.
- [41] *What is a container?* <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>. Accessed 2025-10-25.
- [42] *Pydantic: Data validation and settings management using Python type hints*. Version 2.x, Available: <https://docs.pydantic.dev/latest/>. 2025.
- [43] Ramírez, S. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>. Accessed 2025-10-25.
- [44] Fielding, R. T. “Architectural Styles and the Design of Network-based Software Architectures”. PhD thesis. University of California, Irvine, 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf.
- [45] *Dash Documentation & User Guide*. <https://dash.plotly.com/>. Accessed 2025-10-25.
- [46] *Apache Parquet Documentation*. <https://parquet.apache.org/docs/>. Accessed 2025-10-25.
- [47] *Reading and Writing the Apache Parquet Format (PyArrow)*. <https://arrow.apache.org/docs/python/parquet.html>. Accessed 2025-10-25.
- [48] Montiel, J., Read, J., Bifet, A., Abdesslem, T., and Zliobaite, I. “Scikit-multiflow: A Multi-output Streaming Framework”. In: *Proceedings of the International Conference on Machine*

- Learning (ICML) Workshop on Automated Machine Learning*. 2018. URL: <https://github.com/scikit-multiflow/scikit-multiflow>.
- [49] Looveren, A. V. and Klaise, J. *Alibi Detect: Algorithms for Outlier, Adversarial and Drift Detection*. <https://github.com/SeldonIO/alibi-detect>. Version 0.9.4. 2020.
- [50] AI, E. *Evidently: Evaluate, Test and Monitor Machine Learning Models*. <https://github.com/evidentlyai/evidently>. Version 0.3.2. 2023.
- [51] Duan, Y., Lv, Y., and Wang, F.-Y. “Travel Time Estimation Based on LSTM Neural Network”. In: *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*. 2016, pp. 153–158. DOI: 10.1109/ITSC.2016.7795560.
- [52] Shen, Y., Xiao, Z., Zeng, H., Ye, Y., and Xu, H. “DeepTransport: Learning Spatial-Temporal Dependency for Traffic Condition Forecasting”. In: *IEEE Transactions on Intelligent Transportation Systems* 21.11 (2019), pp. 4607–4617. DOI: 10.1109/TITS.2019.2904751.
- [53] Wang, Y., Chen, J., Zhu, Y., Zhang, Y., Zhang, C., Zheng, Y., and Yu, P. S. “When Will You Arrive? Estimating Travel Time Based on Deep Neural Networks”. In: *AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018. URL: <https://www.aaai.org/ocs/index.php/AAAI/AAAI18/paper/view/16076>.
- [54] Derrow-Pinion, A., Xu, F., Layne, C., and al., et. “ETA Prediction with Graph Neural Networks in Google Maps”. In: *CIKM ’21: Proceedings of the 30th ACM International Conference on Information and Knowledge Management*. 2021, pp. 3767–3776. DOI: 10.1145/3459637.3482246.
- [55] Jia, B., Cao, Y., Song, Z., and Wang, S. “A Spatio-Temporal Graph Convolutional Network for Travel Time Estimation”. In: *IEEE Access* 9 (2021), pp. 129403–129414. DOI: 10.1109/ACCESS.2021.3113696.
- [56] Guo, S., Lin, Y., and al., et. “Learning Dynamic Graph Structure for Traffic Forecasting”. In: *IEEE Transactions on Knowledge and Data Engineering* (2022). DOI: 10.1109/TKDE.2022.3165185.
- [57] Guin, A. *Travel Time Prediction Using Gradient Boosting Decision Trees*. Georgia Institute of Technology Master’s Thesis. 2006.
- [58] Zhang, L. and Haghani, A. “A hybrid model for real-time bus arrival time prediction”. In: *Transportation Research Part C: Emerging Technologies* 60 (2015), pp. 420–435. DOI: 10.1016/j.trc.2015.09.006.
- [59] Antypas, D., Papakostas, A., Georgoudakis, D., and Xydis, S. “Predicting Bus Arrival Time for Autonomous Shuttles in Smart Cities Using Tree-based Models”. In: *2022 International Conference on Smart Applications, Communications and Networking (SmartNets)*. 2022, pp. 1–8. DOI: 10.1109/SmartNets55927.2022.9966784.
- [60] Abhay, A. and Johnson, K. *Automatic Retraining Pipeline for Drift-Adaptation using Evidently and MLflow*. <https://evidentlyai.com/blog/drift-detection-auto-retraining-pipeline>. Under review. 2025.